

This handout includes space for every question that requires a written response. Please feel free to use it to handwrite your solutions (legibly, please). If you choose to typeset your solutions, the `README.md` for this assignment includes instructions to regenerate this handout with your typeset L^AT_EX solutions.

1.a

As seen in the slides, the evaluation function to be used for this case would be depth-limited search.

$$V_{\text{minimax}}(s, d) = \begin{cases} \text{Utility}(s), & \text{if } \text{IsEnd}(s) \\ \text{Eval}(s), & \text{if } d = 0 \\ \max_{a \in \text{Actions}(s)} V_{\text{minimax}}(\text{Succ}(s, a), d), & \text{if } \text{Player}(s) = a_0 \\ \min_{a \in \text{Actions}(s)} V_{\text{minimax}}(\text{Succ}(s, a), d), & \text{if } \text{Player}(s) = a_i, i \in \{1, \dots, n\} \end{cases}$$

The depth is decremented only when the last opponent (n-ghost) has moved and it is Pac-Man's turn again.

3.a

The recurrence for the Expectimax Value function with uniformly random ghosts would be the following:

$$V_{\text{exptmax}}(s, d) = \begin{cases} \text{Utility}(s), & \text{if } \text{IsEnd}(s) \\ \text{Eval}(s), & \text{if } d = 0 \\ \max_{a \in \text{Actions}(s)} V_{\text{exptmax}}(\text{Succ}(s, a), d), & \text{if } \text{Player}(s) = a_0 \\ \frac{1}{|\text{Actions}(s)|} \sum_{a \in \text{Actions}(s)} V_{\text{exptmax}}(\text{Succ}(s, a), d), & \text{if } \text{Player}(s) = a_i, i \in \{1, \dots, n\} \end{cases}$$

In the case of the ghosts, it'd be needed to divide by the number of actions because expectimax computes an expected value under a uniform random policy, not a worst-case outcome as in the minimax situation.

Differently, in the case of Pacman, the action chosen is the best one, and not averaged over the possible actions.

5.a

Minimax always rushes the closest ghost because it assumes the ghosts are adversarial and will choose the action that most harms Pac-Man, making the nearest ghost the immediate worst-case bottleneck. On the other side, expectimax assumes ghosts move uniformly at random and therefore, optimizes expected utility rather than the worst-case outcome.

5.b

By adding a strong penalty in the evaluation when an active ghost is very close to Pacman (so it's about to die), it helps avoid instant death. That makes minimax act more similar to expectimax.

5.c

Let's imagine that we design a spam filter. This filter could be trained with the objective of minimizing the number of emails that users explicitly mark as spam. Because of this, the system is punished mainly when it blocks legitimate emails (false positives), since users are likely to complain when an important or expected email ends up in the spam folder. In contrast, when spam emails are allowed into the inbox (false negatives), users often simply ignore or delete them without reporting them, which produces little or no penalty for the system in comparison to the false positive. As a result, the safest strategy for the model, according to its reward function, would be to allow borderline or suspicious emails to pass through rather than risk blocking legitimate ones. This behavior reduces user complaints and improves the measured objective, but it would increase the overall amount of spam users see, meaning the system optimizes the metric while failing to achieve the designer's true goal of protecting users from spam.